

Resampling Strategies for Model Assessment and Selection

Richard Simon

National Cancer Institute, Rockville, MD, USA.

`rsimon@mail.nih.gov`

8.1 Introduction

The advent of DNA microarrays and proteomics technology has stimulated the development and use of classification algorithms for biomedical studies. In oncology, for example, a common application is predicting response to treatment based on expression profiling of tumor tissue. Such a classifier could be used as an aid in treatment selection for future patients based on the expression profiles of their tumors. In developing such a classifier, it is important to estimate the predictive accuracy that can be expected for future application of the classifier.

There are, in fact, many considerations that may influence the accuracy of predictions made using a classifier developed on a set of data. These include considerations such as assay drift, inter-laboratory variability in assay procedures, tissue handling and preparation, and systematic differences of new cases with regard to unmeasured or unaccounted for prognostic features. The only fully satisfactory way of assessing predictive accuracy of a classifier for independent cases is to classify a set of independent new cases in a manner that reflects all sources of variability to be experienced in broad application of the classifier. Before expending the extensive time and resources necessary for such an “external validation”, one needs a more easily obtained estimate of predictive accuracy that can be obtained from the data set used to develop the classifier.

In this chapter, I will discuss the use of resampling methods to obtain “internal estimates” of predictive accuracy of classification models in applications when the number of candidate predictors (e.g., genes) is much greater than the number of cases (e.g., tumor specimens). Much of the conventional wisdom and many of the published claims about resampling methods are invalid for such settings. For clarity, I will focus on binary classifiers, but the considerations described here apply also to classifiers with more than two classes, and, with some technical modifications, to predicting risk groups using time-to-event data.

Key concepts of this chapter include *classifier*, *true error rate*, *resubstitution estimate*, *resampling estimate*, *bias*, and *variance*. The classifier is the rule for specifying a predicted class as a function of the values of genomic or proteomic measurements. The classifier to be used for subsequent application generally is the one based on the complete set of data in the developmental study. The true error rate is the error rate expected when that classifier is applied to independent data. The re-substitution estimate is the observed error rate when the classifier is applied to the same data set used for its development. The re-substitution estimate is called a biased estimate of the true error rate because the average value of the difference of the two quantities is not zero. A biased underestimate of the true error rate tends to be too small; positive bias means that the estimate tends to be too large. An estimate may be unbiased but have a large variance. That means that, on average, the estimate is not too large or too small, but for any particular set of data the difference between the estimate and the true error rate may be quite large.

8.2 Basic Concepts

8.2.1 Resubstitution Estimate of Prediction Error

Traditionally, statisticians have often used the re-substitution estimate as a measure of predictive accuracy. With the re-substitution estimate, one uses all the data to develop the classifier, and then measures the accuracy of classification on the same set of data used to develop the classifier. The data available for developing the classifier is a set of cases $i = 1, 2, \dots, n$, with each case i consisting of a vector of predictor variables $\mathbf{x}_i$ and a binary 0 or 1 class label y_i . Let D denote the set of data for all n cases, and let $C(\mathbf{x}_i, D) = \hat{y}_i$ denote the class predicted for a case with vector of variables $\mathbf{x}_i$ when the set of data D was used to develop the classifier. Then the re-substitution estimate of error is the proportion of the n cases for which $y_i \neq \hat{y}_i$.

The re-substitution estimate tends to underestimate the true error rate because the same data is used for developing the classifier and for measuring prediction accuracy (Simon et al., 2003). When a model and its parameters are selected to maximize the fit to a set of data, the degree of fit is not a proper estimate of the degree of discrepancy to be expected with independent data. To measure “prediction”, you must separate the data used for developing the classifier from the data used for evaluating performance.

The re-substitution estimate has been frequently used in statistics because the bias is not too great if the number of cases n is large relative to the number of variables p which are measured on the cases and are candidates for inclusion in the classifier. For microarray and proteomic applications, however, the number of candidate variables are generally orders of magnitude greater than the number of cases. In these circumstances, the resubstitution estimate is so biased as to be worthless and completely misleading.

8.2.2 Split-Sample Estimate of Prediction Error

An improved estimate of the true error rate can often be obtained by splitting the data set D into a learning set L and a test set T . For example, the learning set may contain half of the cases and the test set the other half. This is called the *split-sample method*.

A single fully specified classifier is developed on the learning set L . The learning set is not just used to fit the parameters of the classification model, it is used to determine which variables are included in the model and to determine what form of classifier should be used. For example, should the classifier be a linear discriminant, or a neural network with two nodes in a hidden layer, or should it be a nearest neighbor classifier? The learning set is also used to determine all parameters necessary to specify the completely determined binary classifier to be applied to the cases in the test set. I will denote the single fully specified classifier by $C(\mathbf{x}, L)$ to emphasize that only the data in the learning set L can be used in any way for developing the classifier.

Once a single fully specified classifier is developed on the learning set, it is used to classify the cases in the test set. The classifier is not adjusted or calibrated in any way on the test set; it is simply used to classify the cases in the test set and the proportion of cases in the test set incorrectly classified is the estimate of classification error, $\epsilon_{split-sample}$. Let n_T denote the number of test cases. $C(\mathbf{x}_i, L) = \hat{y}_i$ is the predicted class for the i^{th} test case, $\mathbf{x}_i$, made by classifier developed using the learning set. Then

$$\epsilon_{split-sample} = \frac{1}{n_T} \sum_{i=1}^n \delta_i, \quad (8.1)$$

with $\delta_i = 1$ if $y_i \neq \hat{y}_i$ and $\delta_i = 0$ if $y_i = \hat{y}_i$.

There are no well established guidelines for what proportion of the data to use for the learning set and what proportion for the test set, or whether the split should be made randomly or in some systematic manner. Often half to two-thirds of the cases are used for the learning set. The split is often made randomly although in multi-center studies a closer emulation of external validation is obtained if one uses samples from some centers for learning and samples from other centers for testing.

What classifier would be used in an external validation study or for classifying future samples? It would probably be the classifier developed using all the data, i.e., $C(\cdot, D)$ rather than the classifier $C(\cdot, L)$ developed on the learning set. The split-sample process is used to provide an estimate of the classifier $C(\cdot, D)$, although the estimate is based on a classifier developed on less data. If the data set D is large, then the classifier based on the full data set may be about as accurate as that based on the learning set. For small data sets, however, the classifier based only on the learning set may be substantially inferior to that based on the full data and the estimate (8.1) of

prediction error may tend to be too large (i.e., positively biased). Molinaro et al. (2005) evaluated the split-sample method with 50% or 67% of the data used for the learning set and found that for small data sets the split-sample estimate can be seriously positively biased.

Attempting to avoid the positive bias resulting from having too small a learning set may, however, leave such a small test set that the estimate will have a very large variance. Using the split sample method, the number of test set errors will have a binomial distribution and so the variance of the test set error rate is approximately proportional to the reciprocal of the sample size of the test set (Snedecor and Cochran, 1989).

8.3 Resampling Methods

The difficult tradeoffs between the size of the learning set and test set can be somewhat ameliorated by using a large proportion of the samples for the learning set, but repeating the estimate for various learning-testing partitions and averaging the resulting error estimates. Let $(L_1, T_1), (L_2, T_2), \dots, (L_m, T_m)$ denote a set of learning-testing partitions of the data D . For each learning set L_i , develop a classifier from scratch. That means that all aspects of classifier development must be repeated for each learning set, including determination of which features are included in the classifier. The full data set cannot be used for any aspect of the development of the classifiers $C(\cdot, L_1), C(\cdot, L_2), \dots, C(\cdot, L_m)$. Each of these classifiers is applied to the cases not used in its learning set. Let n_{T_j} denote the number of cases in the test set T_j . Then the error estimates are computed as follows:

$$\epsilon_j = \frac{1}{n_{T_j}} \sum_{i=1}^{n_{T_j}} \delta_i, \quad (8.2)$$

with $\delta_i = 1$ if $y_i \neq \hat{y}_i$ and $\delta_i = 0$ if $y_i = \hat{y}_i$. These error estimates are then averaged as follows:

$$\epsilon_{\text{resampling}} = \frac{1}{n_T} \sum_{j=1}^m \epsilon_j \quad (8.3)$$

The resampling estimate is an estimate of the prediction error for the classifier $C(\cdot, D)$ developed on the full data set. It is based on averaging error estimates for a set of different classifiers, one for each learning set generated by the resampling. Resampling methods can only be used when a completely defined algorithm is used for all aspects of the classifier development process. This is because it is not valid to use information from analysis of the full data set in building a classifier from any learning set, and it is not valid to use information from building a classifier in one learning set for building a classifier in a different learning set. This is a serious limitation for large projects where

multiple individuals are involved in the classifier development process and extensive analysis, including biological interpretation, is used in the classifier development process. Consequently, the split-sample method remains a useful approach for many studies.

8.3.1 Leave-One-Out Cross-Validation

A wide variety of different methods have been proposed for constructing the set of partitions $(L_1, T_1), (L_2, T_2), \dots, (L_m, T_m)$ and leave-one-out cross-validation (LOOCV) is one of the most commonly used methods. With LOOCV, the number of partitions m equals the number of cases, n . Each test set consists of a different singleton set; i.e., $T_i = (\mathbf{x}_i, y_i)$, and each learning set consists of all $n - 1$ cases *not* in the corresponding test set; i.e., $L_i = D \setminus T_i$. Since each learning set contains only one fewer cases than the full data set, the positive bias of the split-sample method is almost eliminated. The high variance of the estimate of prediction error for any single test set is reduced by averaging over all of the n learning-testing partitions. Since any two learning sets have $n - 2$ cases in common, there is a high correlation among the models developed on the different learning sets and a high correlation among the error estimates. Although the n estimates of prediction error for the n test sets are almost completely unbiased, they are positively correlated and this correlation serves to increase the variance of the error estimate (Ambroise and McLachlan, 2002). Molinaro et al. (2005) found that for very small samples (e.g., 40 or fewer cases), the large bias of other methods such as the split sample method is more serious than the increased variance of LOOCV. For larger sample sizes, the bias of the split-sample method decreases, and the computational effort of LOOCV also increases. There are some other alternatives, however, that are discussed below.

Although any two learning sets contain $n - 2$ cases in common, it is still essential to repeat the classifier building process from scratch for each learning set. The studies of Ambroise and McLachlan (2002) and of Simon and Lam (2005) showed that failure to re-select the features independently for each learning set results in hugely biased error estimates. Simon et al. (2003) performed a simulation, for example, in which the distribution of expression levels for all genes were completely unrelated to the class indicators. Since the classes were equally represented, any models developed from the data should have a prediction error of about 0.50. The simulations involved 6 000 genes and 14 cases (7 from each class). They used a linear classifier based on the ten genes that seemed most differentially expressed between the classes. They found that if they used all the data to select the ten most differentially expressed genes, to fit the linear model, and then to measure prediction error, the resulting re-substitution estimate was 0 in over 95% of the simulations. This was not so surprising as the re-substitution estimate was known to be biased. It did emphasize, however, just how biased the re-substitution estimate was for settings where the number of candidate variables was much larger than

the number of cases. Simon et al. also evaluated what they called “partial cross-validation”. That is, they used all 14 cases to select the ten genes that seemed most differentially expressed between the classes. They then performed LOOCV, re-fitting the linear classifier to each training set, but always using the same ten genes. What they found was that the partial cross-validation error estimate was zero in over 90% of the simulations. So the partial LOOCV estimate is very misleading, almost as biased as the re-substitution estimate.

Leave-one-out cross-validation is sometimes termed the “jackknife” method. The term “jackknife”, however, has traditionally referred to estimates of bias or variance based on leave-one-out sub-sampling (Efron and Tibshirani, 1993).

8.3.2 k -fold Cross-Validation

The k -fold cross-validation is a popular alternative to LOOCV. Ten-fold cross validation is a special case of k -fold cross validation. With this approach the cases are randomly partitioned into ten disjoint sets of approximately equal size, denoted $T_1, T_2, \dots, T_{10}$. As indicated by the notation, each set forms a test set to be used with the corresponding learning set $L_i = D \setminus T_i$ consisting of the remaining 90% of cases not included in the test set T_i . The classifier development process is repeated ten times, and the resulting classifiers are used to classify the cases in their respective test set to produce error estimates $\epsilon_1, \epsilon_2, \dots, \epsilon_{10}$, which are then averaged as in Equation (8.3). Ten-fold cross-validation requires much less computing than LOOCV when the number of cases is large. It is only slightly more positively biased than LOOCV, since the classifiers are developed on 90% of the full data, and it generally provides error estimates with less variance. Consequently, it is quite popular when there are more than 20 cases. The k -fold cross-validation error estimate can be repeated for different random k -fold partitions and the results averaged. Molinaro et al. (2005), however, found that such averaging had a very limited effect on reducing the variance of the estimate. It can sometimes be useful, however, to design the k -fold partition to be used in a manner that is balanced with regard to a variable correlated with class label.

8.3.3 Monte Carlo Cross-Validation

Monte Carlo cross-validation, also known as *repeated random subsampling*, is a generalization of the split-sample method. It repeats the process of splitting the data into a learning set and a test set, with the model developed on the learning set and the error rate evaluated on the test set. With Monte Carlo cross-validation, the test set estimates are averaged over the learning-testing random splits. For each learning-testing partition, each case appears in either the learning set or the test set, but not in both. Because the learning-testing splits are random, the cases appear in the test sets a variable number of times. That is, one case may appear in m learning sets and in no test set, whereas another case may appear in all test sets, but in none of the learning

sets. With a large number of repetitions, m , however, such extreme differences would be unusual. In k -fold cross-validation, the test sets are disjoint, whereas in Monte Carlo cross-validation, the test sets usually overlap. Molinaro et al. (2005) found that Monte Carlo cross-validation does not reduce the bias of the split-sample method, but it can substantially reduce the variance of the split-sample error estimate. They also reported that ten random learning-testing splits were sufficient to realize most of the achievable reduction in variance.

8.3.4 Bootstrap Resampling

Several versions of the *bootstrap* have been proposed for estimation of the true prediction error. Each bootstrap learning set is composed of n cases selected with replacement from the full set of n cases. Consequently, cases may appear in a learning set multiple times. The bootstrap learning set differs from the learning sets constructed by the other methods in that it contains n cases, albeit not n distinct cases. If L_i denotes the i^{th} bootstrap learning set, the corresponding test set is the set of cases not included in L_i , that is, $T_i = D \setminus L_i$. With the *leave-one-out bootstrap*, the prediction error is estimated as the average of the error rates of the classifiers $C(\cdot, L_i)$ evaluated on the test sets (Efron and Tibshirani, 1993). Although the learning sets contain n cases, since they are not all distinct cases, the leave-one-out bootstrap tends to over-estimate the true error rate.

8.3.4.1 The .632 Bootstrap

To attempt to correct for the positive bias of the leave-one-out bootstrap, Efron and Tibshirani (1993) developed two alternative bootstrap methods. These methods were based on weighted averages of the estimate of the leave-one-out bootstrap and the re-substitution estimate. For the .632 bootstrap, the weight of the leave-one-out bootstrap estimate is 0.632 and the weight for the learning set is 0.368 (Efron and Tibshirani, 1993).

A bootstrapped learning set is generated by randomly sampling n cases uniformly with replacement. Each case has a probability of $p = 1 - n^{-1}$ of *not* being selected. If the data set is sampled n times, then the probability that a case is not selected for the bootstrap sample is $p = (1 - n^{-1})^n \approx e^{-1} = 0.368$ (for large n). Hence, the expected number of distinct cases in the bootstrap learning set is $0.632 \times n$. The error estimate on the test set will be pessimistic, since the model is trained on 63.2% of the data only. Therefore, the error estimate on the learning set is combined with the error estimate test set with different weights. If b bootstrapped data sets are generated, then the estimate for the classification error is $\epsilon_{.632}$:

$$\epsilon_{.632} = .632\epsilon_{loobs} + (1 - 0.632)\epsilon_{resub} \quad (8.4)$$

$$\epsilon_{loobs} = \frac{1}{b} \sum_{i=1}^b \epsilon_{T_i} \quad (8.5)$$

$$\epsilon_{resub} = \frac{1}{b} \sum_{i=1}^b \epsilon_{L_i}, \quad (8.6)$$

where ϵ_{T_i} is the observed error rate on the i^{th} test set and ϵ_{L_i} is the observed error rate on the corresponding i^{th} learning set

The .632 bootstrap estimate can be very downward biased, however, for high-dimensional data (Molinario et al., 2005). For example, consider the situation where the expression data is uninformative for predicting the class variable. In that case the true prediction error is 0.5 for equally represented classes, and the leave-one-out bootstrap is unbiased. The leave-one-out bootstrap is unbiased because there is really no penalty for developing classifiers based on a reduced number of distinct cases in the learning set, since no classifier that performs better than the flip of a coin is possible. Even in this situation, however, the re substitution estimate can be close to zero as shown by (Simon et al., 2003). Since the .632 bootstrap estimate is a weighted average of the leave-one-out bootstrap estimate and the re-substitution estimate, the result is also downward biased.

8.3.4.2 The .632+ Bootstrap

Efron and Tibshirani (1997) developed the .632+ bootstrap to attempt to improve on the bias of the .632 bootstrap. With the .632+ bootstrap, the weight for the leave-one-out bootstrap is not a fixed value but adjusted based on an estimate of the degree to which the data is overfit.

$$\epsilon_{.632+} = w \cdot \epsilon_{loobs} + (1 - w)\epsilon_{resub} \quad (8.7)$$

$$w = \frac{.632}{1 - 0.368R} \quad (8.8)$$

$$R = \frac{\epsilon_{loobs} - \epsilon_{resub}}{\epsilon_{random} - \epsilon_{resub}} \quad (8.9)$$

The quantity R is supposed to provide an estimate of the degree of overfitting. ϵ_{random} is an estimate of the expected error when the class labels are independent of all prediction variables. Molinario et al. (2005) found that the .632+ bootstrap performed well except for high-dimensional data in cases where the classes were well separated. In those cases, the .632+ estimate could be much greater than the true value.

8.4 Resampling for Model Selection and Optimizing Tuning Parameters

Many classification algorithms contain one or more tuning parameters that must be specified before a completely specified binary classifier is obtained. For example, linear support vector machine classifiers with inner product kernels contain two parameters that must be determined. One is equivalent to a ridge regression shrinkage parameter and the other represents the relative cost of misclassification of cases in the two classes. The *shrunk centroid classification algorithm* of Tibshirani et al. (2002) contains a parameter which determines the amount of shrinkage, and ultimately the number of variables used for classification. We might represent these classifiers by $C(\mathbf{x}, \lambda_i, D)$ indicating that one has a set of classifiers, indexed by the values of the tuning parameters λ_i . Often resampling methods, such as those described above, are used to obtain estimates of the prediction error for a set of such classifiers. Let us denote the true prediction error for the classifier $C(\mathbf{x}, \lambda, D)$ by $\tau(\lambda_i, D)$ and the resampling estimator by $\epsilon(\lambda_i, D)$. It is reasonable to select a vector of tuning parameters to be used for future classification as that which minimizes the estimated error rate; that is:

$$\lambda^* = \arg \min_{\lambda_i} \{\epsilon(\lambda_i, D)\} \quad (8.10)$$

It is important to recognize, however, that the resampling estimate of prediction error at λ^* is not an unbiased estimate of the prediction error that can be expected for the selected classifier $C(\mathbf{x}, \lambda^*, D)$. That is, $\epsilon(\lambda^*, D)$ is not an unbiased estimator of $\tau(\lambda^*, D)$. A bias is created in the process of minimizing over the set of tuning parameters. Varma and Simon (2006) have studied this bias and indicated that its size depends on the number of classifiers minimized over and the variability of the resampling estimates. The process of optimizing over values of the tuning parameters, however, should be viewed as an integral part of the algorithm for determining a completely specified classifier. As emphasized above for the case of selecting genes, it is the entire algorithm for determining a completely specified classifier which needs to be embedded in the resampling algorithm. In order to obtain a proper estimate of $\tau(\lambda^*, D)$, one should re-select the “optimal” tuning parameter vector for each resampled learning set. Since the selection of an optimal tuning parameter involves the use of a resampling estimator, one needs to implement a *two-fold nested resampling* in order to accomplish this. The *outer loop* of resampling is for the purpose of estimating the prediction error of a completely specified classifier. The *inner loop* of resampling is for the purpose of defining a completely specified classifier by optimizing the tuning parameters (Varma and Simon, 2006). In the inner loop, the learning set is split into a *training set* and a *validation set*, as illustrated in Figure 8.1.

The resampling produces multiple classifiers, constructed on multiple training sets and tested on their associated validation sets. This process is,

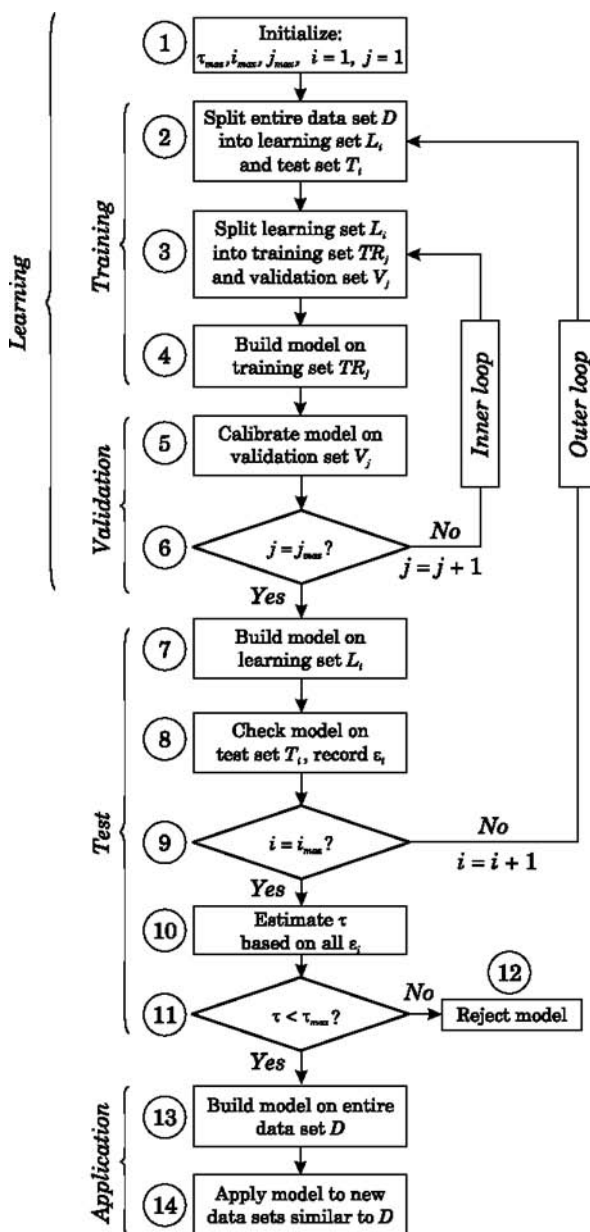


Fig. 8.1. Basic modeling process involving learning (training and validation), testing, and application.

however, just a statistical procedure to estimate the prediction accuracy of the model fitted to the full data set. Data analysts are sometimes tempted to use the similarities and differences among the classifiers developed on the resampled training sets for purposes of model selection. This happens particularly often with regard to feature selection. That is, a defined feature selection is used for classifier development in each training set. The analyst is often surprised at how much the sets of selected features differ among the training sets and they are tempted to use for the final classifier only those features that were consistently selected in most training sets. That, however, represents a completely new algorithm for classifier development. When one is using resampling for estimating the true error rate of a classifier, then all aspects of the algorithm used for developing the classifier must be repeated in each resampled learning set. If part of the algorithm involves determining which variables to select based on their frequency of selection in resampling the data, then the resampling process must be performed for variable selection for each learning set of the resampling loop used for error estimations.

8.4.1 Estimating Statistical Significance of Classification Error Rates

In addition to providing a fair estimate of prediction accuracy for a classifier, it is usually important to test the null hypothesis that the prediction accuracy is no better than one could have obtained by chance. One can address this question by obtaining the distribution of resampled error estimates for data sets in which there is no relationship between the candidate prediction variables and the class labels. Such data sets can be generated by randomly permuting the assignment of class labels to cases, keeping fixed the number of cases for each class. For each permutation D^* , the resampling procedure is completely repeated resulting in a new classifier $C(\mathbf{x}, D^*)$ and a new error estimate $\epsilon_{\text{resampling}}(D^*)$. This is done for either all possible permutations or for a large number of random permutations, and the area in the left tail of this distribution corresponding to error rates no greater than that obtained with the un-permuted labels $\epsilon_{\text{resampling}}(D)$ represents the significance level of the test of the null hypothesis. This approach was first described by Radmacher et al. (2002). Michiels et al. (2005) proposed an alternative test based on whether a prediction interval for the resampled prediction error estimate excluded 0.5, but it is less powerful because its inefficient use of the data in training-validation splits.

8.4.2 Comparison to Classifiers Based on Standard Prognostic Variables

Another question that often arises is whether the genomic or proteomic data enables better predictions than were already possible using standard clinical or histopathological variables. Some investigators attempt to address this by

using resampling to obtain a class prediction for each case, and using these predictions in a classification model that involves the predictions and the other variables. For example, let $\hat{y}_i$ denote the predicted class for case i based on a classifier developed using a learning set not containing case i . One might then study a logistic regression model containing these genomic or proteomic predictions as well as other variables. The problem is, however, that although the predictions were based on learning sets not containing the cases predicted, the predictions are generally not statistically independent and so the resulting logistic inference is not valid. These problems have been noted by Tibshirani and Efron (2002).

Kattan (2003, 2004) points out that prognostic models should be judged based on their ability to predict, not based on issues of the significance of regression coefficients for some variables after adjustment for other variables.

8.5 Comparison of Resampling Strategies

Molinaro et al. (2005) conducted a detailed study of different resampling methods for estimating prediction error in the context of high-dimensional data where the number of candidate predictors is much larger than the number of cases. They evaluated the methods with several different classification methods including diagonal linear discriminant analysis, linear discriminant analysis, nearest neighbor classification, and CART classification trees. They examined the resampling methods and classification algorithms in the context of data sets of different sizes, different numbers of informative features and different signal strengths (class separation relative to noise). For small data sets (e.g., 40 cases), they found that the split sample methods with one-half to two thirds of cases used for training produced very biased over-estimates of prediction error. Monte Carlo repeats of the split samples had no effect on the strong bias. For such data sets they found that LOOCV, 10-fold cross-validation and the .632+ bootstrap generally had the least bias and smallest mean-square error. The .632+ bootstrap was more biased than LOOCV and 10-fold cross-validation, however, in small data sets with strong signals. The differences among resampling methods decreased as the sample size increased.

8.6 Tools and Resources

Investigators and data analysts need to exercise caution in use of packaged software because much of it does not use complete resampling methods essential for high-dimensional data. *BRB-ArrayTools* software incorporates all of the resampling methods described in this chapter for a wide variety of classifier types. The software is available without cost for non-commercial applications (Simon and Lam, 2005).

8.7 Conclusions

Resampling-based estimates of prediction error are not substitutes for fully independent validation studies, but are essential for helping to identify when independent validation studies are warranted.

The re-substitution estimate commonly employed is highly biased for genomics and proteomics applications and should not be used. The split-sample method is useful for large studies in which many analyses and many individuals are involved in model development. The test data should not be used in any way until a single, completely specified classifier is developed and agreed to by the collaborators. At that time the classifier should be applied to classify the cases in the test set, with no calibration or adjustment of the model. The prediction accuracy of the classifier is scored on the test cases.

For smaller projects the split-sample approach represents an inefficient use of the data compared to resampling methods. Resampling methods, however, require that the classifier development process be algorithmic and repeatable in its entirety on multiple learning sets.

Feature selection is an integral part of classifier development and must be repeated from scratch for each learning set when resampling is used.

If resampling is used to optimize tuning parameters of a classifier, then that resampling should be viewed as an integral part of classifier development and the entire algorithm should be embedded in an outer resampling loop used to estimate the prediction accuracy of the resulting classifier.

Resampling methods provide an estimate of the prediction error that can be expected for future use of the classifier fit to the entire data set. It is the classifier developed on the full data set that is taken forward. The resampling produces multiple classifiers, constructed on the multiple training sets and tested on their associated validation sets. This process is, however, just a statistical method to estimate the prediction accuracy of the model fitted to the full data set, the intermediate models should not be used in the definition of the classifier to be taken forward.

For small data sets, leave-one-out cross-validation or 10-fold cross validation provide almost unbiased estimates of prediction error for a variety of classifiers and a wide variety of signal strengths. The mean-square error of these methods compares favorably to those of sample-splitting based methods that provide too few cases in the training set and thereby result in over-estimates of prediction accuracy. For larger data sets, 10-fold cross-validation is preferable to LOOCV with regard to computational effort and variance. The standard leave-one-out bootstrap and .632 bootstrap can be quite biased for small data sets. The .632+ bootstrap is generally competitive with LOOCV and 10-fold cross-validation for small data sets except those with very strong signals.

References

- Ambroise, C. and McLachlan, G.J. (2002). Selection bias in gene extraction on the basis of microarray gene expression data. *Proc. Natl. Acad. Sci. USA*, 98:6562–6566.
- Efron, B. and Tibshirani, R. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
- Efron, B. and Tibshirani, R.J. (1997). Improvements on cross-validation. The .632+ bootstrap method. *J. Am. Stat. Assoc.*, 92:548–560.
- Kattan, M.W. (2003). Judging new markers by their ability to improve predictive accuracy. *J. Natl. Cancer Inst.*, 95(9):634–635.
- Kattan, M.W. (2004). Evaluating a new marker's predictive contribution. *Clin. Cancer Res.*, 10:822–824.
- Michiels, S., Koscielny, S., and Hill, C. (2005). Prediction of cancer outcome with microarrays: A multiple random validation strategy. *The Lancet*, 365:488–492.
- Molinaro, A.M., Simon, R., and Pfeiffer, R.M. (2005). Prediction error estimation: A comparison of resampling methods. *Bioinformatics*, 21(15):3301–3307.
- Radmacher, M.D., McShane, L.M., and Simon, R. (2002). A paradigm for class prediction using gene expression profiles. *J. Comp. Bio.*, 9(3):505–511.
- Simon, R. and Lam, A.P. (2005). BRB-ArrayTools Users Guide (version 3.3). Technical Report 28, Biometric Research Branch, National Cancer Institute, Bethesda, MD, USA.
- Simon, R., Radmacher, M.D., Dobbin, K., and McShane, L.M. (2003). Pitfalls in the analysis of DNA microarray data: Class prediction methods. *J. Natl. Cancer Inst.*, 95:14–18.
- Snedecor, G.W. and Cochran, W.G. (1989). *Statistical Methods*. Iowa State University Press, Ames Iowa, USA.
- Tibshirani, R., Hastie, T., Narasimhan, B., and Chu, G. (2002). Diagnosis of multiple cancer types by shrunken centroids of gene expression. *Proc. Natl. Acad. Sci. USA*, 99(10):6567–6572.
- Tibshirani, R.J. and Efron, B. (2002). Pre-validation and inference in microarrays. *Stat. Appl. Gen. Mol. Biol.*, 1(1).
- Varma, S. and Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91.